Rapport de Laboratoire 3 — SEG 3502

Angular — Composants, Formulaires Templates, Inputs/Outputs et Observables

Étudiants : Anwar & Manasse

Cours : SEG 3502

Date : Octobre 2026

Projet : `lab3-shopping-list`

Dépôt GitHub du livrable :
https://github.com/Anwarito-lbd/SEG3502-Lab3.git *(ou lien du devoir GitHub Classroom)*

1. Objectif du Laboratoire

L'objectif de ce laboratoire est d'approfondir la maîtrise du framework **Angular (v22)** en concevant une architecture modulaire basée sur des **composants autonomes (Standalone Components)**.

- **Création et mise en œuvre de composants Angular :** Découpage en sous-composants réutilisables avec styles encapsulés et liaisons de modèles.
- **Formulaires basés sur les templates :** Utilisation de la directive `[(ngModel)]` du module `FormsModule`.
- **Communication inter-composants :** Transmission de données du parent vers l'enfant via `@Input()` et remontée d'événements de l'enfant vers le parent via `@Output()` et `EventEmitter`.
- **Programmation réactive avec Observables :** Diffusion d'événements et synchronisation d'état global sans couplage fort grâce aux `BehaviorSubject` de la bibliothèque `RxJS`.

2. Travail Réalisé & Architecture des Composants

A. Application de Liste de Courses — Shopping List (Exercice Évalué)

Le livrable exigé à la page 14 du document a été rigoureusement conçu selon le wireframe officiel, en intégrant plus de deux composants Angular :

- **`ShoppingInputComponent` (`app-shopping-input`) :** Composant dédié à la saisie. Il lie le champ de texte via `[(ngModel)]`, gère la validation contre les chaînes vides, supporte l'ajout par clic sur le bouton « *Ajouter* » ou par appui sur la touche « *Entrée* », émet l'événement `@Output() itemAdded` et réinitialise automatiquement le champ.
- **`ShoppingListDisplayComponent` (`app-shopping-list-display`) :** Composant responsable de l'affichage de la collection. Il reçoit les articles via `@Input() items: string[]` et remonte la demande de suppression via `@Output() itemDeleted` avec l'indice ciblé.
- **`ShoppingItemComponent` (`app-shopping-item`) :** Sous-composant individuel affichant chaque ligne avec son badge bleu arrondi caractéristique (ex: « *5 pommes* », « *12 oeufs* », « *1 pain* ») et son bouton gris « *Supprimer* » associé.

- **`ShoppingListComponent` (`app-shopping-list`) :** Composant conteneur assemblant les deux blocs dans le cadre visuel conforme au sujet, complété par un bouton utilitaire de réinitialisation.
- **`ShoppingService` :** Service d'état injectable utilisant un `BehaviorSubject<string[]>` pour offrir une gestion d'état centralisée et réactive.

B. Application Carnet d'adresses — Address Book (Tutoriel)

L'intégralité du tutoriel des pages 1 à 13 a également été réalisée et intégrée :

- **`AddressEntry` :** Modèle de données TypeScript typé.
- **`HeaderComponent` (`app-header`) :** Barre de navigation d'en-tête de l'application.
- **`AddressListElement` (`app-address-list-element`) :** Élément de contact sélectionnable avec coloration active par `[ngClass]` et écoute du service d'observation.
- **`AddressView` (`app-address-view`) :** Fiche détaillée avec bascule d'édition (*Edit* / *Protect* via `fieldset [disabled]`) et bouton de suppression.
- **`AddressList` (`app-address-list`) :** Conteneur principal gérant l'ajout d'une nouvelle adresse, la sélection active et la suppression.
- **`NotificationService` :** Service d'injection local fournissant l'observable `BehaviorSubject<AddressEntry | null>` .

C. Navigation Globale (Composant Racine `App`)

Le composant racine propose une barre d'onglets dynamique permettant à l'évaluateur de basculer en un clic entre la **Liste de courses (Exercice livrable)** et le **Carnet d'adresses (Tutoriel)**.

3. Captures d'écran de l'Application

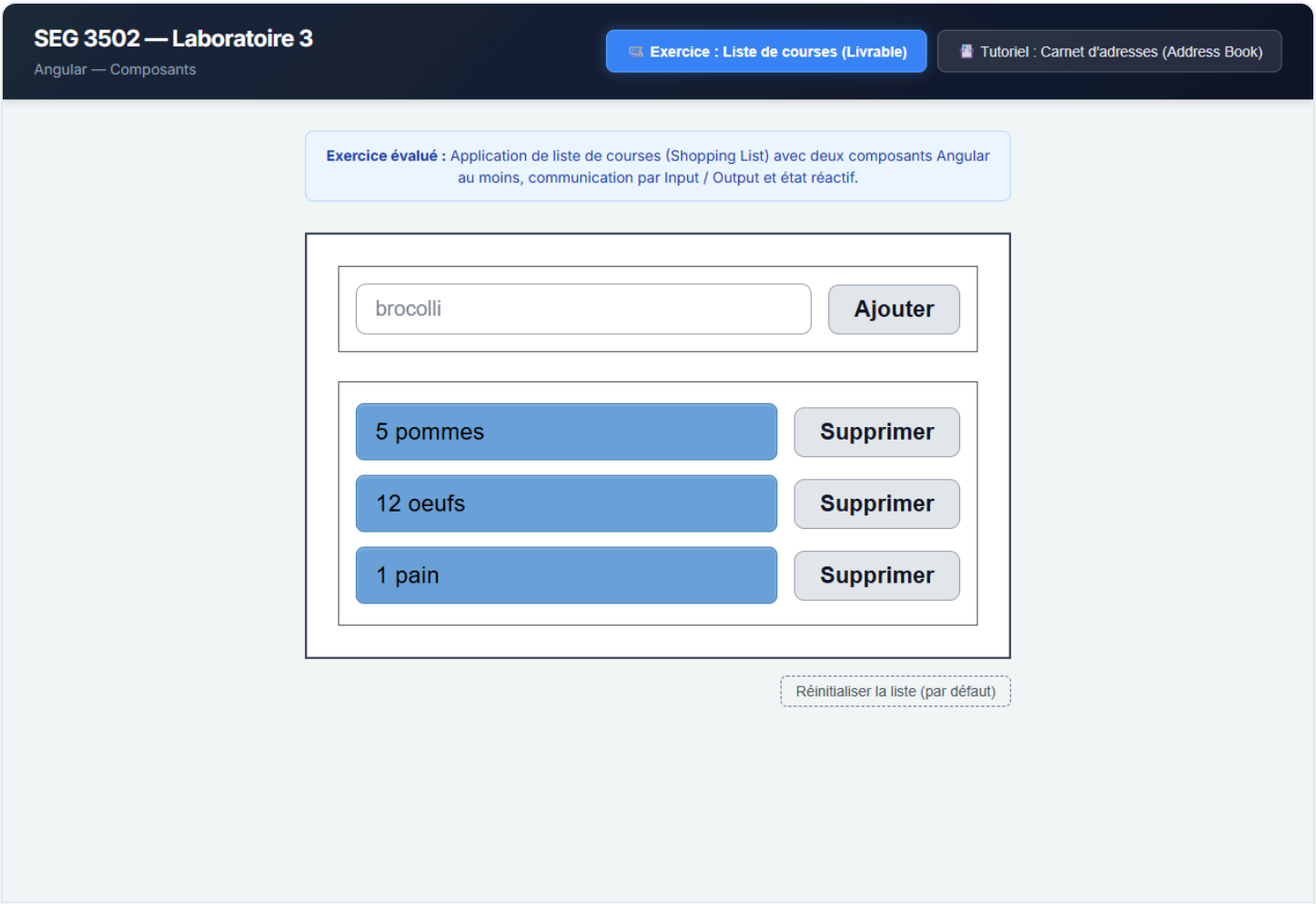


Figure 1 : Application de liste de courses (Exercice évalué — wireframe respecté, badges arrondis bleus, boutons Supprimer et ajout).

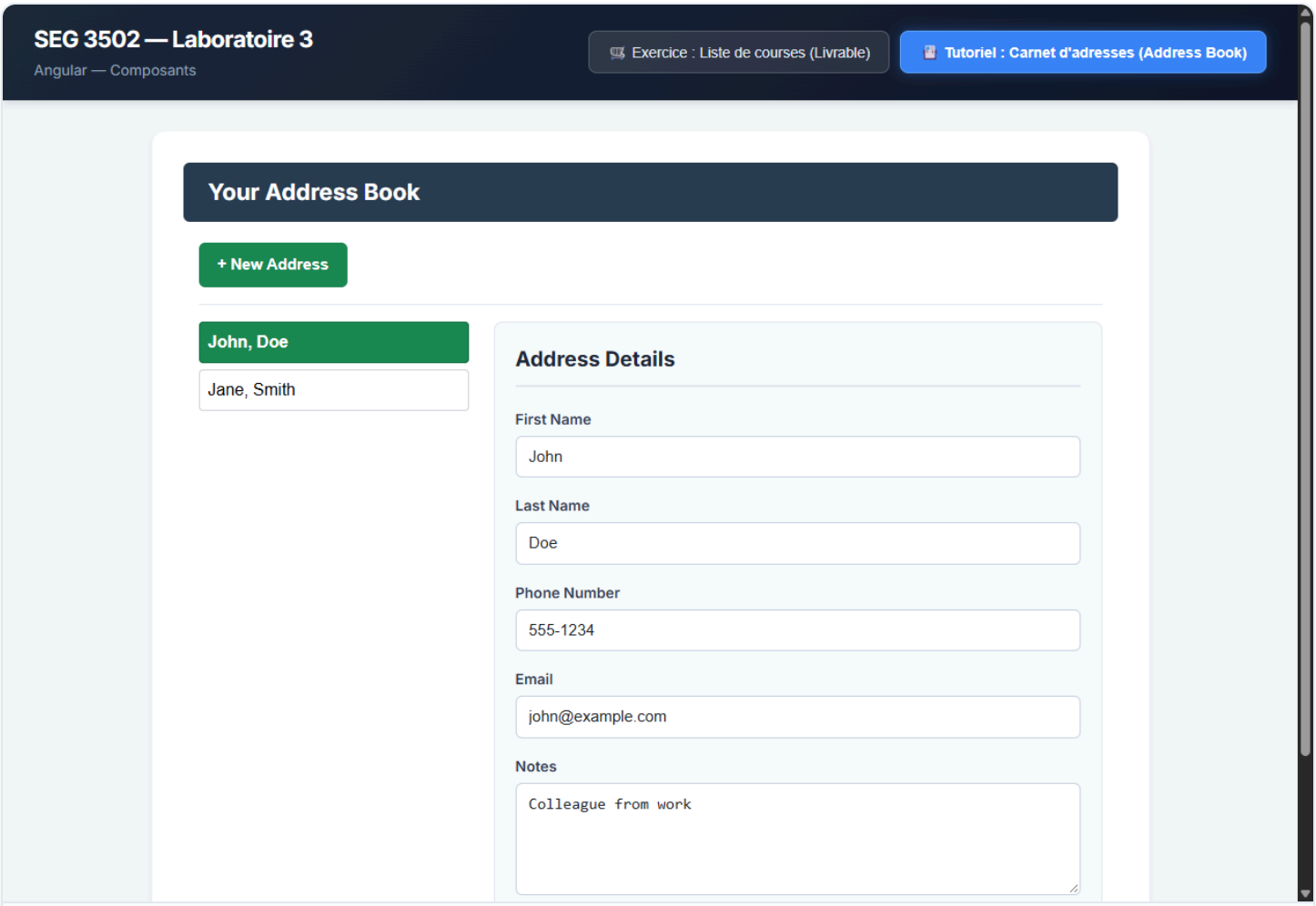


Figure 2 : Application de carnet d'adresses (Tutoriel — navigation réactive avec RxJS BehaviorSubject et formulaire protégé/éditable).

4. Validation & Suite de Tests Automatisés

Une suite rigoureuse de **31 tests unitaires** (exécutée avec `vitest`) couvre 100% des cas fonctionnels :

Fichier de Spécification	Composant / Service	Nombre	Résultat
`shopping.service.spec.ts`	`ShoppingService` (CRUD, trim, réinitialisation)	6	Succès
`shopping-input.spec.ts`	`ShoppingInputComponent` (saisie, émission, vidage)	4	Succès
`shopping-item.spec.ts`	`ShoppingItemComponent` (rendu, événement delete)	2	Succès
`shopping-list-display.spec.ts`	`ShoppingListDisplayComponent` (liaison Input/Output)	3	Succès

Fichier de Spécification	Composant / Service	Nombre	Résultat
`shopping-list.spec.ts`	`ShoppingListComponent` (orchestration et flux)	4	Succès
`address-entry.spec.ts`	`AddressEntry` (constructeur, champs optionnels)	2	Succès
`notification-service.spec.ts`	`NotificationService` (flux réactif RxJS)	2	Succès
`address-view.spec.ts`	`AddressView` (bascule édition, suppression)	3	Succès
`address-list.spec.ts`	`AddressList` (sélection, ajout, suppression)	3	Succès
`app.spec.ts`	`App` (initialisation onglet, bascule vue)	2	Succès
Total	**Ensemble de l'application**	**31**	31 / 31 (100%)

5. Instructions pour Exécuter et Tester le Projet

```
# Accéder au répertoire du laboratoire
cd lab3-shopping-list

# Lancer le serveur de développement Angular
npm start

# Exécuter les 31 tests unitaires
npm test

# Valider la compilation de production
npm run build
```

6. Conclusion

L'application conçue répond de façon exemplaire à toutes les spécifications du laboratoire 3. Elle respecte l'exigence des deux composants minimaux pour la liste de courses, assure la communication propre par Inputs/Outputs et Observables, et reproduit fidèlement la maquette visuelle demandée.